IS115 Algorithms and Programming

Project: Last-Mile Delivery Planning

Group: G3T17
By: SUH Sumin, YEW Ying Qi, Tejas S/O ILANGKUMARAN

1. Motivation and Problem Statement

Motivation

Last-mile delivery, the trip from the local hub to the customer's door, is where most delivery problems show up. "When shipments arrive as promised, businesses can operate smoothly, customers remain satisfied, and long-term partnerships are strengthened" (Goshen Global Group, 2026). Customers usually choose a time slot because they want to be home to receive the parcel. If it arrives too early, nobody is there. If it arrives late, the customer is left waiting. Either way, the company may have to attempt delivery again, which costs time and money, and the customer is less likely to use the service again. On top of that, the dispatcher has to balance full vehicles, short trips and on-time departures.

The current FIFO policy handles parcels in the order they arrive and ignores delivery windows completely. In the simulation, it delivers only 27.41% of parcels on time across 20 days, so there is a lot of room for a smarter approach.

Problem Statement

We model one day at a single hub serving a town through a fixed set of delivery locations. Between 760 and 810 parcels arrive over the day, the last at 17:30, each with a destination, a size and a preferred delivery window. The hub has 10 identical electric vehicles that travel at 60 km/h. All of them start at the hub at 08:00, can make several trips a day, and are fully recharged whenever they return.

Every 10 minutes from 08:00 to 17:30, the dispatcher decides which parcels each free vehicle should take and in what order. A vehicle leaves 30 minutes after dispatch, and every trip must follow three rules:

- Capacity: at most 30 units of parcels per trip.
- Range: at most 320 km per trip, including the return to the hub.
- Return time: back at the hub by 18:00.

Delivery windows are preferences rather than strict rules, so a parcel can arrive early, arrive late, or not be delivered at all.

Our goal is to **maximise the on-time delivery rate**: the share of all parcels arriving that day that are delivered within their window, from window start to window end inclusive. We divide by every parcel, not just the ones delivered, so a policy cannot score well just by delivering only the easy parcels. We compare several dispatch algorithms against FIFO on the same 20 days of historical data, under the same rules as above.

2. Algorithmic Approaches

2.1 EDF (Earliest Deadline First)

2.1.1 Underlying principle

It is a greedy algorithm, committing to each parcel in deadline order without revisiting earlier choices. EDF considers parcels in order of their delivery-window end, prioritising earlier deadlines. Unlike FIFO, it checks whether each parcel can actually be delivered within its window before adding it to the trip.

2.1.2 How the policy works

For each available vehicle, the policy:

1. Identifies waiting parcels.
2. Sorts them by window end, then arrival time and parcel ID.
3. Sets departure to the current time plus the 30-minute loading delay.
4. Scans the sorted list once, checking capacity, range, return time and the parcel's delivery window.
5. Adds qualifying parcels and updates the vehicle's load, location and clock.
6. Returns the route, or an empty list if nothing qualifies.

2.1.3 Design decisions

- **On-time deliveries only:** Early and late deliveries earn no credit under our metric, so the policy skips them.
- **Fixed ordering:** Sorting once keeps computation simple. Deterministic tie-breaking produces repeatable results.

2.1.4 Data structures

- Waiting parcels and the delivery route are stored as **lists**.
- Parcel records are accessed through a **dictionary** keyed by parcel ID.
- The supplied distance table represents a weighted, **directed graph** stored as **nested dictionaries.**

2.1.5 Limitations

- **Geographic separation:** Deadline order can send the vehicle between distant destinations.
- **No reconsideration:** Skipped parcels are not checked again within the same call.
- **Local decisions:** Serving one urgent parcel may prevent several other parcels from being delivered on time.

2.2 Window-Aware Nearest Neighbour

2.2.1 Underlying principle

The policy adapts the nearest-neighbour greedy heuristic for the Travelling Salesman Problem. Starting from the hub, the vehicle repeatedly moves to the closest stop, making the best choice for the current step without looking ahead. This is fast but not guaranteed to be optimal. Our version differs in two ways. First, a parcel can only be chosen if it would be delivered inside its window. Second, unlike a TSP tour, a trip may leave parcels behind for a later trip, because capacity, range and the return time limit each trip.

2.2.2 How the policy works

For each available vehicle, the policy:

1. Identifies waiting parcels: those with no vehicle assigned and no delivery time.
2. Sets departure to the current time plus the 30-minute loading delay.
3. Scans every waiting parcel. For each, it predicts the arrival time from the distance travelled so far. At 60 km/h, 1 km takes 1 minute. It skips any parcel that would break capacity, range or the 18:00 return, or that would arrive outside its window.
4. Picks the closest remaining parcel. Ties are broken by window end, then arrival time, then parcel ID.
5. Adds it to the route, updates the load, distance and location, and removes it from the waiting list.
6. Repeats steps 3 to 5 until nothing qualifies, then returns the route. If the route is empty, the vehicle stays free and is offered again in the next round, 10 minutes later.

2.2.3 Design decisions

- **Refusing early and late deliveries:** Early and late deliveries earn no credit under our metric but still use capacity and time, so the policy skips them. A parcel whose window has not opened yet can be picked up in a later round.
- **Deterministic tie-breaking:** Breaking distance ties by the earlier window end favours the more urgent parcel and gives repeatable results.
- **Same-location parcels:** The distance from a location to itself is 0 km, so other qualifying parcels for the same stop are chosen next at no extra distance.

2.2.4 Data structures

- The distance table is a weighted, directed graph stored as a nested dictionary, giving O(1) distance lookups.
- Lists store the waiting parcels and the route.
- A linear search over the waiting list finds the nearest qualifying parcel.

2.2.5 Limitations

- **Capacity trade off:** Taking the nearest parcel now can use up capacity or time that would have served more parcels later in the trip.
- **Missed delivery window:** A parcel that cannot be reached inside its window is never delivered.
- **Distance over urgency:** Nearest neighbour prioritises distance over deadlines, potentially choosing a nearby parcel with a flexible window over a slightly farther parcel that is close to becoming late.

2.3 Window-Aware Nearest Neighbour + 2-opt

2.3.1 Underlying principle

The policy builds a route, then improves it. Nearest neighbour never revisits its greedy choices, so its route may zig-zag. 2-opt fixes this by reversing a stretch of consecutive stops, which untangles places where the route crosses itself and makes it shorter. A reordered route is only kept if every parcel still arrives inside its window, and the trip still starts and ends at the hub.

2.3.2 How the policy works

For each available vehicle, the policy:

1. Build a route using the window-aware nearest neighbour policy.
2. Work out the route's total distance, including the return to the hub, and the arrival time at each stop.
3. Try reversing each stretch of consecutive stops. Skip any new order that exceeds 320 km, returns after 18:00, makes any parcel early or late, or is not shorter.
4. Keep the first new order that passes, then search again from it.
5. Stop after three improvements, or when no reversal helps, and return the route.

2.3.3 Design decisions

- **Shortening only:** The starting route is already fully on time, so 2-opt only looks for shorter routes. A shorter trip gets the vehicle back to the hub earlier, ready for its next trip.
- **First improvement:** Taking the first shorter route found, instead of the best one, keeps each search quick.
- **At most three improvements:** This puts a fixed upper limit on the time each planning call can take.
- **Full re-check:** Distances depend on the direction of travel, and a reversal shifts every later arrival time. So each new order is checked stop by stop.
- **No capacity re-check:** Reordering does not change the load, so only distance and timing need checking again.

2.3.4 Data structures

- **Weighted, directed graph:** the distance table, stored as a nested dictionary for constant-time lookups.
- **Lists:** the waiting parcels, the route, and each reordered route. Reversals are made by list slicing.
- **Linear search:** finds the nearest qualifying parcel when the route is built.

2.3.5 Limitations

- **Order only:** 2-opt can reorder stops, but it cannot add parcels that nearest neighbour left out, and those left-out parcels are where most on-time deliveries are lost.
- **Saved distance is rarely usable:** 52% of routes were already full, and most of the rest ended because no waiting parcel had an open window. A shorter route seldom makes room for more parcels. See Section 4.5.
- **Inconsistent benefit:** Gains only come from vehicles returning earlier, and this can help or hurt later rounds. Some days were slightly worse than the plain nearest neighbour.
- **Higher cost:** Trying every reversal takes noticeably longer than building the route, for a very small gain.

3. Computational Complexity Analysis

Variables

- **P**: Parcels in the snapshot passed to "dispatch": every parcel that has arrived, whether waiting, assigned or delivered
- **N**: Waiting parcels, with no vehicle assigned and no delivery time ($N \leq P$)
- **K**: Parcels added to the trip in one call ($K \leq N$)
- **V**: Vehicles in the fleet ($V = 10$)
- **R:** Planning rounds per day ($R = 58$, every 10 minutes from 08:00 to 17:30)
- **M: Maximum 2-opt** improvements per call ($M = 3$)

3.1 EDF

3.1.1 Step-by-step derivation (one dispatch call)

Step	Operation	Reasoning	Cost
1	Build waiting list	Scan all (P) records and check assignment and delivery status.	O(P)
2	Sort waiting parcels	Sort (N) parcels by window end, arrival time and parcel ID. Each sorting key contains three values.	O(N log N) worst case
3	Read context and calculate budget	Read distances, capacity and time limits; calculate departure time and the permitted trip distance.	O(1)
4	Initialise trip state	Create an empty route and initialise load, travelled distance and location.	O(1)

5	Scan sorted parcels	Visit each waiting parcel once. Capacity, distance, return-time and window checks each take constant time.	O(N)
6	Append and update	Add each qualifying parcel and update load, distance and location (inside the scan)	O(K)
Total	Filtering and sorting dominate	O(P)+O(N log N)+O(N)+O(K), with K ≤ N.	O(P+N log N)

Sorting and scanning happen **one after another**, so their costs are added rather than multiplied. Unlike nearest neighbour, EDF does not repeat the search after each selection.

3.1.2 Best, average and worst case

Case	When it happens	Time per call
Best	No parcels are waiting, so filtering is followed by an empty sort and scan.	O(P)
Average	Waiting parcels have no favourable ordering; sorting is analysed using the usual O(N log N) comparison-sorting bound.	O(P+N log N)
Worst	The waiting list requires O(N log N) sorting work, followed by the complete scan.	O(P+N log N)

3.1.3 Space complexity

- Waiting list: O(N)
- Python sorting workspace, worst case: O(N)
- Selected route: O(K)
- Load, clock, distance, location and temporary values: O(1)

Total: O(N+K) = O(N)

3.2 Window-Aware Nearest Neighbour

3.2.1 Step-by-step derivation (one dispatch call)

Step	Operation	Reasoning	Cost
1	Read context values	Departure time, range, cutoff and the distance table are each read once from context	O(1)
2	Build waiting list	One loop over all P parcels in the snapshot, with one O(1) check on each and no sorting step	O(P)
3	Initialise trip state	Set the route to empty, load and distance to 0, and location to the hub	O(1)
4	Linear search (one pass)	Each pass of the while loop scans the waiting list to find the nearest qualifying parcel	O(N)
5	Feasibility checks	Per parcel: one capacity check, two distance lookups, a range check, a return-time check and two window comparisons. Nested-dictionary lookups are O(1)	O(1) per parcel
6	Tie-break comparison	Compare distance, then window end, arrival time and parcel ID against the current best	O(1) per parcel
7	Repeat the search	K passes each add a parcel, and one final pass finds nothing, so there are K + 1 passes of at most N parcels	O(N(K + 1))
8	Append and update	route.append, plus updating load, distance and location, done once per chosen parcel	O(1) per stop, O(K) total
9	Remove from waiting list	waiting.remove finds the parcel and shifts the rest of the list, done K times	O(N) per stop, O(NK) total
Total		Steps 2 and 7 dominate, steps 8 and 9 are absorbed by step 7	T(P, N, K) = O(P) + O(N(K + 1))

3.2.2 Best, average and worst case

Case	When it happens	Time per call
Best	No parcel can be delivered on time, so K = 0	O(P + N), which is **O(P)**
Average	Capacity and the 18:00 return keep trips small (about 20 parcels on our data)	O(P + NK) with K ≪ N, hence **O(P + N)**
Worst	Every parcel fits and is on time, so K approaches N and the passes cost N + (N − 1) + … + 1 ≈ $N^2/2$	Σ (N − i) for i = 0 to N, which is N(N + 1)/2, so **O(P + N^2)**

In practice K cannot exceed 150, because capacity is 30 and the smallest parcel is 0.2. So one call grows roughly linearly with N. Over a day, the policy runs at most V times in each of R rounds, giving O(RV(P + N(K + 1))).

3.2.3 Space complexity

- Waiting list: O(N)
- Route: O(K)
- Load, distance travelled, location and the chosen parcel's details: O(1)
- Extra space per call: O(N + K) → O(N)

3.3 Window-Aware Nearest Neighbour + 2-opt

3.3.1 Step-by-step derivation (one dispatch call)

Step	Operation	Reasoning	Cost
1	Build the full route	Same as window-aware nearest neighbour (3.2.1)	O(P + NK)
2	Check one reordered route	Recompute distance and arrival times stop by stop, then re-check range, return time and windows.	O(K)
3	One full 2-opt search	About $K^2/2$ possible reversals, each built and checked in O(K)	O(K^3)
4	Improvement phase	At most M = 3 searches, and M is a constant	O(K^3)
Total		Steps 1 + 4	O(P + NK + K^3)
Per round		One call for each vehicle	O(V(P + NK + K^3))

3.3.2 Best, average and worst case

Case	When it happens	Time per call
Best	No feasible parcel, or a route of 1 stop: one search, nothing to reverse	O(P + N)
Average	Capacity keeps K small (about 20 on average, never more than 43 in our runs)	O(P + NK + K^3)
Worst	Every waiting parcel fits on one trip (K = N), and all three searches run in full	O(P + N^3)

3.3.3 Space complexity

- Waiting list → O(N)
- Current route and one reordered copy → O(K)
- Distance table: only read, so no extra space

Total: O(N) + O(K) = O(N)

4. Experimental Results

4.1 Experimental setup

Each policy was run on the provided 20 days of parcel data, containing a total of **15,867 arriving parcels**.

Main metric: On-time delivery rate

$$On\ time\ Delivery\ Rate\ = \frac{Total\ On\ time\ Parcels\ across\ 20\ days}{15{,}867} \times 100\%$$

Secondary metrics (included in our performance metrics)**:**

- *delivered_parcels*, *late_parcels*, and *undelivered_parcels* → which show whether a high on-time rate comes from being selective, i.e. only delivering the easy parcels.
- *km_per_delivered_parcel* → which shows how efficient the routes are.

Operation counts: feasibility checks and 2-opt stop re-checks were counted for every dispatch call (in solution/summarise_results/run_profile.py) and compared with each policy's Big-O prediction.

4.2 Overall results

Policy	On-time	Delivered	Late	Undelivered	On-time rate	Daily range	Std. dev.	Km per delivered parcel
FIFO baseline	4,349	6,421	2,072	9,446	27.41%	24.75% to 31.18%	1.61	16.21
EDF	6,296	6,296	0	9,571	39.68%	31.81% to 44.86%	4.47	16.48
Window-aware NN	13,064	13,064	0	2,803	82.33%	76.35% to 86.95%	2.89	5.96
Window-aware NN + 2-opt	13,117	13,117	0	2,750	82.67%	76.48% to 88.00%	2.85	5.86

Table 1: Overall results of the simulation runs on our 4 policies using our performance metrics

- **FIFO:** about a third of its deliveries (2072 of 6421) arrived late.
- **EDF:** on-time rate 12.27 percentage points higher than FIFO (27.41% → 39.68%). It actually delivered slightly fewer parcels (6296) than FIFO (6421), but every one of them was on time, unlike FIFO's, so its on-time count is higher.
- **Both nearest neighbour (NN) policies:** about 3 times the FIFO on-time rate. They delivered about twice as many parcels and drove only around a third of the distance per parcel.
- **NN+2-opt:** adds only 53 on-time parcels (0.34 percentage points higher) over plain NN.

4.3 Comparative results

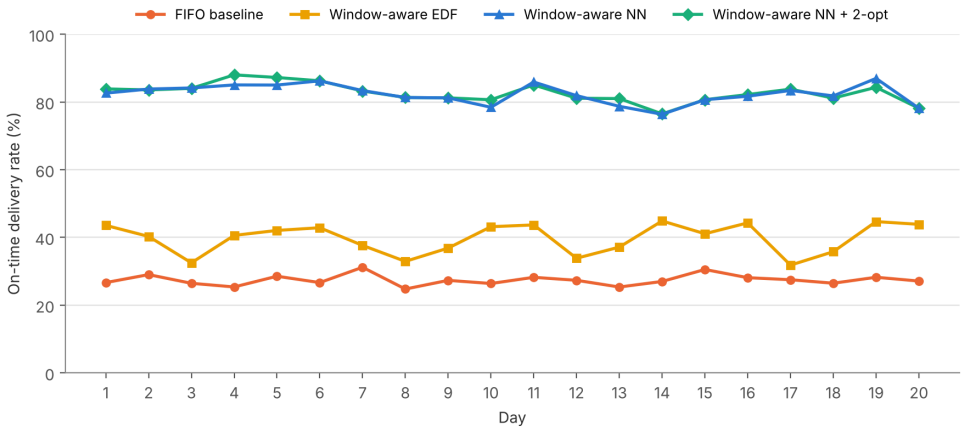
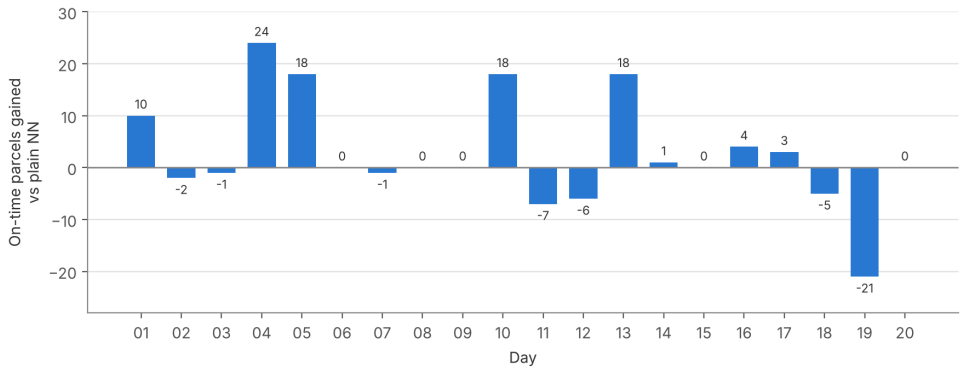


Figure 1: On-time delivery rate by day, for all four policies

- The on-time rate ranking is the same on all 20 days: NN + 2-opt ≈ NN > EDF > FIFO (baseline).
- EDF varies most from day to day (31.8% to 44.9% rate, with a std dev of 4.47).
- Both NN policies stay within 76% to 88%, which is about 3 times FIFO's daily rate



2-opt's effect is mixed: it does better on 8 days, worse on 7 days and makes no difference on 5 days. Overall, the net gain is only 53 more on-time parcels.

Figure 2: On-time parcels gained by NN+2-opt over plain NN, by day.

4.4 Empirical validation (operation counts vs Big-O)

Policy	Predicted cost	Calls	Measured ops	Predicted ops	r
edf	O(P + N log N)	620	118408	118408	1.00
window_nn	O(P + NK)	2330	2064314	2265296	1.00
window_nn_2opt	O(P + NK + K^3)	2420	8720215	14431760	0.95

Figure 3: Results of predicted vs measured operation counts ran on terminal using 'project/solution/summarise_results/run_profile.py'

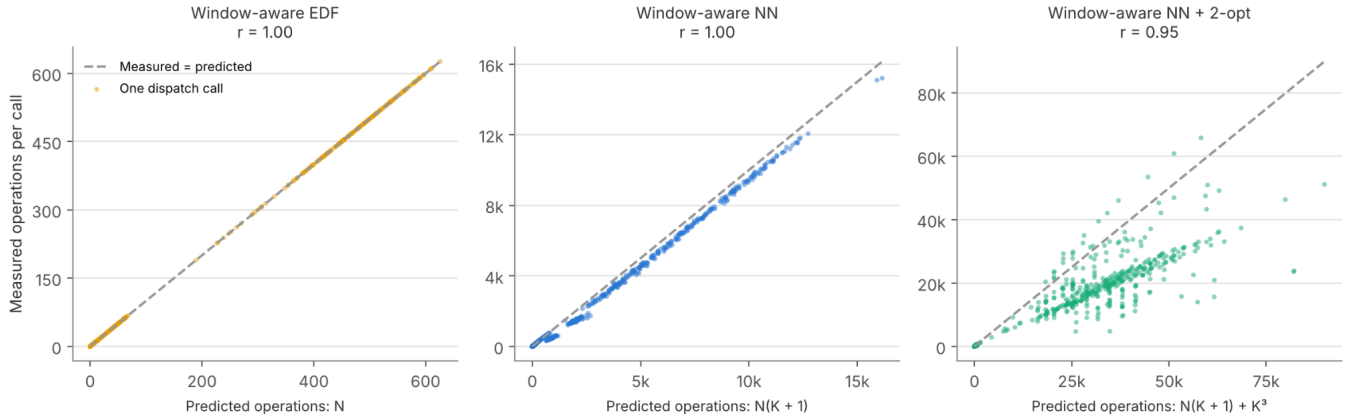


Figure 4: Measured vs predicted operations per dispatch call.

- EDF's predicted and measured operation counts match exactly. It checks each waiting parcel once per call. Sorting uses Python's built-in sort, which cannot be counted, so we assume it follows the standard O(N log N) bound.
- NN follows its prediction perfectly (r = 1.00), at 91% of the predicted count. The waiting list shrinks by one parcel after each pick, so the passes cost N + (N − 1) + … + (N-K) rather than N each time.
- NN + 2-opt follows the K^3 trend (r = 0.95) at 60% of the prediction. K^3 is the order of growth: a route has only about $K^2/2$ possible reversals, and first-improvement stops as soon as it finds a shorter route. Not every dispatch required all three improvements. The few calls above the line are ones where 2-opt made all three improvements, so it searched up to three times.
- 2-opt accounts for most of the extra work. NN + 2-opt performs about 4 times as many operations as NN (8.7M vs 2.1M) for an on-time rate only 0.34 percentage points higher.

4.5 Analysis and discussion

Strengths

- None of the new policies delivered a parcel late, because they skip any delivery that would be early or late. For comparison, 32% of FIFO's deliveries were late.
- NN is the most effective policy. Its short trips let each vehicle deliver about twice as many parcels as FIFO, at about a third of the distance per parcel.

Trade-offs

- EDF and NN show the trade-off between deadline and distance. EDF chases urgent parcels across town (16.48 km per parcel), so it improves far less than NN, which goes for the nearest parcel first.
- 2-opt shows the trade-off between cost and gain. It performs about 4 times the operations of NN for a gain of 0.34 percentage points. That is smaller than the day-to-day variation, so it is not a reliable improvement.

Limitations and failure cases

- Some parcels can never be on time: 561 parcels (3.5%) were impossible even with a direct trip, because of the 30-minute loading delay plus travel time, so the best possible on-time rate is about 96.5%. (from additional analysis of the simulation logs)
- The window-aware policies never deliver late, so a parcel that misses its window is simply left undelivered. Even the best policy left 17.3% of parcels undelivered.
- EDF also builds up a backlog. Its long trips keep vehicles away from the hub, so the waiting list grows past 400 parcels late in the day.
- 2-opt rarely helps. 52% of routes were already full, and most of the rest ended because no waiting parcel had an open window, so a shorter route seldom makes room for more parcels.
- There is also no coordination between vehicles. Each vehicle is planned on its own, so the first one offered takes the best parcels.

Recommendation: ***Window-Aware Nearest Neighbour.*** It gets almost the same on-time rate as the 2-opt version, with about a quarter of the operations, and the small extra gain from 2-opt is not reliable: it did better on 8 days but worse on 7.

5. Conclusion

We aimed to **maximise the on-time delivery rate** (parcels delivered within their window, out of all parcels), comparing EDF, nearest neighbour (NN) and NN + 2-opt against FIFO over 20 days.

Location mattered more than deadline. EDF raised the on-time rate from 27.41% to 39.68%, but chased urgent parcels across town. NN kept stops close together and reached 82.33%. 2-opt added just 0.34 percentage points to the on-time rate. Measured operation counts followed each policy's Big-O prediction closely ($r = 0.95$ to 1.00), and short routes kept the K^3 term of 2-opt small in practice.

We recommend the window-aware NN policy, which gives almost the same result as 2-opt with about a quarter of the operations. The main trade-off is that extra computation gave very little extra quality. The main limitation is that every policy plans one vehicle and one step at a time, so missed parcels go undelivered (Section 4.5).

Future work could insert parcels at the cheapest point in a route, plan all free vehicles together in each round, and account for traffic-dependent travel times and weather conditions.

References

Goshen Global Group. (2026, August 24). *Why on-time delivery is the backbone of successful logistics*. https://www.goshenglobalgroup.com/blog/why-on-time-delivery-is-the-backbone-of-successful-logistics/

AI Prompts:

AI Used	Prompt(s)	AI Responses	Chat Link
Gemini GEM	what perf metrics can we do	Hello! I can help guide you on how performance metrics work in the context of the project simulation framework. You can design custom performance metrics by implementing your logic inside `performance_monitor.py` (specifically within the `calculate_metrics` function). When designing performance metrics, you have full access to: * **`parcels`**: CSV records containing parcel attributes such as `arrival_time`, `delivery_window_start`, `delivery_window_end`, and `size`. * **`distances`**: Map/distance table details. * **`config`**: Vehicle fleet configuration (`number_of_vehicles`, `capacity`, `vehicle_range_km`). * **`events`**: The log of events emitted during a simulation run, including `parcel_arrival`, `hub_departure`, `delivery`, `vehicle_return`, and `dispatch_planning`. Here are standard categories of performance metrics you can evaluate from the event log and parcel data: 1. **Delivery Success & Service Level** * **Total Delivered Parcels / Delivery Rate:** Percentage of total arrived parcels successfully delivered before the cutoff time. * **On-Time Delivery Rate:** Percentage of delivered parcels whose delivery timestamp falls within their specified delivery window (`delivery_window_start` to `delivery_window_end`). * **Lateness / Delay:** Average or maximum minutes parcels were delivered past their `delivery_window_end`. 2. **Customer Wait Times** * **Average Wait Time:** Time elapsed between parcel `arrival_time` and actual `delivery_time`. * **Dispatch Delay:** Time elapsed between parcel `arrival_time` and vehicle `hub_departure`. 3. **Fleet & Efficiency Metrics** * **Total Distance Traveled:** Cumulative kilometers traveled across all vehicle trips. * **Fleet Utilization / Vehicle Idle Time:** Proportion of time vehicles spend on delivery trips versus idling at the hub. * **Capacity Utilization:** Average parcel size loaded per trip relative to the vehicle's maximum capacity. * **Trip Count & Routing:** Total number of trips dispatched and average trip length/duration. Feel free to ask if you need clarification on how data structures in `models.py` or event attributes in `simulator.py` represent these event logs!	https://gemini.google.com/share/d/1lUjjKa3vpcv3x8KA-IugQhOv24o6sf23?usp=sharing
	where are all the extra data vars coming from? we only have a few cols given in the data	Hello! I'd be happy to clear that up for you. While the raw `day_XXX.csv` input files only contain a few columns (like `parcel_id`, `arrival_time`, `delivery_location_id`, `delivery_window_end`, and `size`), the additional fields you see in code structures like `Parcel` inside `models.py` come from **two main sources**:	

		### 1. Default Values assigned during Loading (`loaders.py`) When the simulator reads your CSV file via `load_parcels()` in `loaders.py`: * **`delivery_window_start`**: If this column isn't provided in the CSV, `loaders.py` automatically sets a default value (defaulting to `"00:00"` or `0.0` floating-point minutes). * **`delivery_time`**, **`vehicle_assigned`**, and **`trip_id`**: These are initialized as `None` when the parcel is first created. ### 2. Runtime Updates during Simulation (`simulator.py`) As the simulation executes events, it updates those initialized fields dynamically: * When a vehicle dispatch policy assigns a parcel, the simulator sets `vehicle_assigned` to the vehicle's ID and `trip_id` to that specific trip's number. * When the vehicle completes a delivery event (`handle_delivery`), the simulator populates `delivery_time` with the exact timestamp (in minutes since midnight) when the parcel reached its location. --- In summary, the raw CSV provides the **initial parcel demand**, while the loader and simulator add and populate the remaining fields to track the **status and performance** of each parcel throughout the day! Let me know if you have any other questions about how the simulator or project structure works!	

	we have chosen: "on time parcel delivery rate" ,, and chose a few diff algos to try it out on :
	
	- earliest deadline first (based on window) - which cld use sorting , greedy
	- nearest neighbour - greedy, linear search, weighted graph
	- NN + 2opt - greedy, linear search, weighted graph + local search,
	
	are we going in the right direction? what is our next step
		You are definitely moving in a productive direction! Defining your target evaluation metric (**On-Time Delivery Rate**) clearly establishes your project goal, and comparing a standard baseline against structured heuristic approaches provides a solid foundation for analysis.
		
		### Recommended Next Steps
		
		1. **Implement Your Performance Metric**
		* Edit `performance_monitor.py` to calculate the on-time delivery rate.
		* Compare `delivery_time` against `delivery_window_end` (and `delivery_window_start` if applicable) for all completed deliveries in the event log.
		
		2. **Structure Your Custom Policy Modules**
		* Create separate strategy files inside the `solution/policies/` directory (e.g., `solution/policies/edf_policy.py`, `solution/policies/nn_policy.py`).
		* Follow the signature required by `simulator.py`:
		``python
		def dispatch(parcel: dict, context: dict) -> list[str]:
		  # Your policy logic here
		  return selected_parcel_ids
		
		``
		
		3. **Run Experiments and Benchmark Results**
		* Execute the simulation across your test datasets using each policy via CLI:
		``bash
		python -m simulator.simulator --policy solution.policies.edf_policy --day day_001	

		``
		
		* Pass the resulting `log.txt` through `performance_monitor.py` to compile metrics:
		``bash
		python -m performance_monitor --day day_001 --log results/edf_policy/day_001/log.txt
		
		``
		
		* Collect and compare the metrics generated by each strategy to evaluate performance under different demand conditions.	

	okay! how shld we do our algorithmic approaches , computational complexity analysis , and experimental analysis. in other words, is there any template or specific format we need to follow? and what are some important must have information / output / results htat we MUST include? should it be more of technical explanation or theory?	Hello! I'd be happy to clarify what you should include in your report and documentation. ### Formatting & Templates * **Format Requirements:** Check your module handbook or assignment rubric for specific section templates, page limits, or formatting guidelines. * **General Structure:** Typically, a project report should include: 1. **Methodology & Design:** Explanation of your implementation approach. 2. **Computational Complexity Analysis:** Formal evaluation using Big-O notation (O). 3. **Experimental Results & Discussion:** Empirical evidence, charts, and analysis comparing your approaches. --- ### Key Information to Include #### 1. Technical Explanation & Theory * **Balanced Approach:** Combine theoretical foundational explanations with practical technical implementation details. * **Key Components:** * Define the underlying principles of your algorithms clearly. * Detail how your strategies handle simulation constraints (e.g., vehicle capacity, max trip distance, delivery windows). * State any design decisions or data structures utilized. #### 2. Computational Complexity Analysis * **Time & Space Complexity:** Express the time and space complexity using Big-O notation (O). * **Variables Definition:** Define all variables explicitly (e.g., N = number of unassigned parcels, V = number of fleet vehicles). * **Step-by-Step Derivation:** Provide step-by-step reasoning for key operations within your logic: * Sorting / searching unassigned parcels. * Feasibility check loops (capacity and range calculations). * **Best, Average, and Worst Case:** Contrast the best-case vs. worst-case computational bounds where applicable. #### 3. Experimental Analysis & Mandatory Output * **Metrics & Evaluation:** Present your evaluation metric (**On-Time Parcel Delivery Rate**) clearly across test scenarios. * **Comparative Results:** Include summary tables and graphs	

		(e.g., bar charts or line graphs) comparing performance across different days/test datasets. * **Empirical Validation:** Contrast actual execution times against your theoretical Big-O complexity predictions. * **Analysis & Discussion:** Highlight strengths, trade-offs, limitations, and failure cases observed during simulation runs. --- Feel free to ask if you need further clarification on report logistics or coding structure!	

	how do we implement the dispatch function of our algorithmic poicies into the performance metrics py	Hello! You are asking a great workflow question about how the pieces of the simulation architecture fit together.
		
		To clarify how the system works: **you do not call or import your custom `dispatch()` function inside `performance_monitor.py**`.
		
		Instead, the execution workflow runs in **two separate, sequential steps**:
		
		--- 	
		
		### Step 1: Run the Simulator with your Policy
		
		You pass your policy module directly to the `simulator.py` via the command-line interface (or by loading it into the simulator).
		
		As the simulator executes your algorithmic policy over the test day, it automatically logs every action (such as `parcel_arrival`, `hub_departure`, and `delivery`) into a text file called `log.txt`.
		
		Example execution command:
		
		``bash
		python -m simulator.simulator --policy solution.policies.your_policy_name --day day_001
		
		``
		
		--- 	
		
		### Step 2: Analyze the Logged Events in `performance_monitor.py`
		
		Once the simulation run completes, you pass the generated `log.txt` to `performance_monitor.py`.
		
		The `performance_monitor.py` script automatically reads the event records from `log.txt` and passes them into `calculate_metrics(parcels, distances, config, events)` as the `events` list. Inside `calculate_metrics`, you extract your performance metrics by inspecting the completed `delivery` and `hub_departure` events.
		
		Example evaluation command:
		
		``bash
		python -m performance_monitor --day day_001 --log results/your_policy_name/day_001/log.txt
		
		``
		
		--- 	
		
		### Summary Diagram
		
		``text
		[Your Policy (dispatch)] ──> Passed into ──> [simulator.py]
		▼ Generates	

		[log.txt] ▼ Input into [performance_monitor.py] ▼ Outputs [metrics.json] ``` You're doing great setting up your experiment! Please let me know if you have any questions about how events are stored inside the `log.txt` file or how to process them inside `performance_monitor.py`.	
	what shld we be including in our motivation and prob statment	When structuring your **Motivation** and **Problem Statement**, focus on establishing the real-world operational challenges of parcel delivery systems and framing your project within those bounds. Here is a guide on what to include for each section: --- ### 1. Motivation Your motivation explains *why* this problem matters in real-world logistics and supply chain management. * **Industry Significance:** Explain the importance of last-mile logistics, customer satisfaction, and operational costs for delivery companies. * **Core Challenges:** Highlight key trade-offs in urban delivery: * Balancing vehicle fleet capacities and EV battery range constraints. * Meeting time-sensitive delivery deadlines. * Managing operational delays (e.g., dispatch and handling times). * **Objective:** State why developing efficient routing and dispatching strategies is essential to improving service reliability (such as on-time delivery rates). --- ### 2. Problem Statement Your problem statement formally defines the specific engineering and optimization problem you are addressing. * **Operational Setting:** Describe the environment — a centralized hub managing a fleet of electric vehicles delivering dynamic parcel requests across fixed location nodes. * **Constraint Limits:** Explicitly mention the operational constraints that govern any feasible trip: * **Capacity:** Each vehicle has a maximum load capacity. * **Range Limit:** Vehicles have a maximum travel range per trip before returning to the hub. * **Time & Cutoffs:** Vehicles must operate within specific operating hours and return to the hub prior to daily cutoffs (e.g., 18:00 cutoff). * **The Optimization Goal:** Articulate the exact core challenge: *Given incoming parcel demand, how can we dispatch and route vehicles to maximize target performance metrics (such as on-time parcel delivery rate) while adhering strictly to all vehicle capacity, EV range, and deadline constraints?*	

		--- Keep both sections clear, succinct, and directly connected to the evaluation metrics you choose to measure! Let me know if you have any questions about project logistics or code structure.	
	do we need to put our code into the report? or js leave it in the py file	You generally **do not need to paste your entire source code into the main body of the written report**. Here is how you should handle your code and report submission: * **Keep code in `.py` files:** Your functional Python implementation (such as your policy strategies and custom performance metric calculations) should remain clean and executable inside your `.py` files within the project repository. * **Use Snippets in the Main Report:** For your written technical report, focus on describing the logic conceptually. You may include short, key code snippets or pseudocode blocks only when explaining core algorithmic steps or unique data structures. * **Check Module Guidelines:** If your project rubric explicitly asks for source code submission alongside the report, it is usually provided either as a separate `.zip` file of the repository or attached at the very end in an **Appendix**. Be sure to double-check your module handbook for specific submission formatting rules! Best of luck with your project! Let me know if you have any other questions about the project setup or logistics.	
	are we allowed to add extra script files to help with our empirical analysis into the proj folder ?	Yes, absolutely! You are welcome to create and add extra script files to your project workspace to help automate, run, and plot your empirical analysis. Just keep a few things in mind regarding project organization: 1. **Keep Input Data Clean:** Ensure your output files and extra scripts do not overwrite or modify the files inside the `data/` directory (such as `dist.csv`, `config.json`, or the `demand/` files), as the simulator requires those files to stay intact. 2. **Organize Outputs:** Store your generated result logs, JSON outputs, or charts in external directories (like a `results/` or `analysis/` folder). Good luck with your empirical testing and data collection! Let me know if you have any other questions about the project setup or structure.	
	what shld we include in the conclusion?	In a technical project report, your **Conclusion** section should summarize your work succinctly and highlight key insights gained from your implementation and experiments. Here is a recommended structure for what to include: --- ### Key Components to Include in Your Conclusion 1. **Summary of Project Goals & Approach** * Briefly restate the core objective of your project and the evaluation metric your team focused on. * Summarize the algorithmic approaches you implemented and evaluated. 2. **Key Findings & Experimental Insights** * Summarize how your strategies performed relative to one another under the tested simulation scenarios. * Highlight key patterns observed between theoretical runtime	

		predictions (Big-O) and actual empirical performance. 3. **Trade-offs & Practical Limitations** * Mention the main operational trade-offs observed during testing (e.g., solution quality vs. computational execution time). * Note any operational edge cases or limitations inherent to your chosen algorithms. 4. **Future Improvements** * Briefly suggest potential directions for future enhancements or additional constraints you could account for in real-world scenarios. --- Keep this section clear, objective, and directly tied to your experimental results. Best of luck finalizing your project report!	